CipherVault

Confidential Evidence Storage for Reporting Sexual Violence on Campus

Frenaldy Bestabba Hasugian (NRP 5025241156) · I Made Ryan Darmajaya (NRP 5025241159) · Muh. Aqil Alqadri Syahid (NRP 5025241161) · Kadek Andra Wikanjaya Putra (NRP 5025241187)

Information Security / IUP — Assignment 1 · Repository: https://github.com/andra-wp/testing

1. Scenario: Reporting Sexual Violence on Campus

Victims of sexual violence and harassment on campus often do not report it because they fear that their identity, and what happened to them, will spread beyond the people handling the case. CipherVault gives a student one place to hand evidence to the campus unit that handles such reports (the **admin**), while keeping that evidence unreadable to anyone without the reporter's passphrase — including whoever obtains the database or a copy of an evidence file.

There are two roles. A **reporter** registers with a self-chosen username (no real name is required), encrypts evidence, keeps the returned encrypted envelope and can decrypt it again. The **admin** sees the list of submitted evidence and its metadata and can download any envelope, but can open one only with the passphrase that the reporter chose; the victim therefore decides if and when the content is disclosed. A report holds chat screenshots, an identity document (KTM/KTP, only if the reporter chooses to be identified), office documents and video, plus notes (Table 1). Leaking any of these exposes the victim, the perpetrator and witnesses and can lead to retaliation and re-victimisation. E-mailing the files to a staff member leaves unencrypted copies in several mailboxes and forwards; here every file is encrypted before it is stored, in one place, behind role-based access.

1.1. Which data is private

Data	What it contains here	Private?	Basis (GDPR / UU PDP)
Account & notes	pseudonym; chronology naming people, places	Yes	GDPR Art. 4(1), 9(1) (sex life); UU PDP Pasal 4(3)
Chat screenshots	names, numbers, messages of victim and others	Yes	GDPR Art. 4(1) (incl. third parties); UU PDP Pasal 4(3)
Identity doc (KTM/KTP)	NIK, name, photo, address	Yes	UU PDP Pasal 4(3); photo not used for identification, so not biometric (GDPR Recital 51)
Office docs	chronology, medical report	Yes, specific	health: GDPR Art. 9(1); UU PDP Pasal 4(2)(a)
Video	faces, voices, intimate content	Yes, specific	GDPR Art. 4(1), 9(1) (sex life)

Table 1: Classification of stored data. Treated as **not** private: cipher parameters (algorithm, mode, salt, nonce, MAC), which are public by design (Kerckhoffs), and operational metadata (file size, timestamp). GDPR Art. 32(1)(a) and Art. 5(1)(f) name encryption and integrity as security measures for such data.

2. Design and Justification

2.1. Architecture

A React + Vite frontend talks to a FastAPI backend backed by SQLite (`instances/ciphervault.db`). *Upload:* the browser sends the file and passphrase; the server derives the key, encrypts, stores the envelope in `evidences.envelope_json` and returns it to the reporter. *Review:* the admin lists evidence and downloads envelopes. *Download:* envelope + passphrase are sent back; the server checks the MAC, decrypts and returns the file. Encryption runs on the server because browsers' WebCrypto offers AES but neither DES nor RC4; client-side encryption would need hand-written JavaScript for two ciphers, whereas the server uses one maintained library (PyCryptodome) and one key-derivation path for all three. The cost is that the server sees plaintext and passphrase during a request, so it must be trusted and served over HTTPS (§4).

2.2. Ciphers and modes of operation

AES-256 — CTR. CTR turns the block cipher into a stream: no padding (no padding-oracle risk, ciphertext as long as the file), a corrupted bit affects only that bit, and blocks are independent, which suits large videos. Its one requirement, never reusing a nonce under a key, holds because every file gets a fresh key (fresh salt) and a random 8-byte nonce (64-bit counter). CTR is malleable, so we add HMAC-SHA256 (encrypt-then-MAC). We did not use GCM because DES has no GCM mode; one construction (CTR + HMAC) serves both block ciphers.

DES — CTR. Same reasons as AES. With an 8-byte block, the 4-byte nonce leaves a 32-bit counter: 2^{32} blocks × 8 B = 32 GiB per file, far above the 100 MB upload limit; per-file keys keep each key far from the 64-bit-block birthday bound. The 56-bit effective key remains brute-forceable (§4).

RC4. A stream cipher with no mode or nonce: keystream reuse is prevented only because each file's key is new (fresh salt → fresh PBKDF2 output). Its keystream biases remain (we do not discard initial bytes), and it is prohibited in TLS (RFC 7465).

2.3. Key handling

Key = PBKDF2-HMAC-SHA256(passphrase, 16-byte random salt per file, 600 000 iterations, 64 B): the first 32 B (8 B for DES) are the cipher key, the last 32 B the HMAC key, so encryption and integrity never share a key. 600 000 iterations is the OWASP recommendation for PBKDF2-HMAC-SHA256; its measured cost is 114 ms per file (§3). Salt, nonce and MAC are stored in the envelope; the passphrase, derived keys and plaintext are never stored. A forgotten passphrase makes the evidence unrecoverable, even for the admin: we accept this rather than keep a master key that could be stolen or compelled.

2.4. Storage

Each upload is one `evidences` row: owner id, category, filename, algorithm, mode, notes, size, envelope and timestamp. The envelope is JSON with Base64 fields (≈ 1.33 × the file size, measured). Filename, category and notes are kept in plaintext, both in columns (for the admin list) and inside the envelope; for this scenario this is the weakest point of the current implementation (§4). Files are read fully into memory, so uploads are capped at 100 MB.

2.5. Authentication, access control and integrity

Account passwords are hashed with PBKDF2-HMAC-SHA256 (600 000 iterations, 16-byte random salt) and compared with `hmac.compare_digest`. Login returns a random 256-bit token held in server memory and sent as a Bearer header; every crypto endpoint requires it and `/api/admin/*` additionally requires the `admin` role. Data comes back intact because the server recomputes HMAC-SHA256 over nonce ‖ ciphertext and compares it in constant time **before** decrypting: a wrong passphrase or a modified ciphertext is rejected instead of silently yielding garbage, which plain CTR would do.

3. Comparative Analysis of AES, DES and RC4

3.1. What we measure and why

A reporter waits for each upload, an admin for each opening, and the university pays for storage. Per realistic evidence size we measure (1) cipher-only time, to compare the ciphers themselves; (2) end-to-end envelope encryption/decryption exactly as the API runs it (PBKDF2 + cipher + HMAC + Base64/JSON), which is what users wait for; (3) PBKDF2 alone; (4) envelope size. Network time is excluded: it is identical for all ciphers.

3.2. Methodology

13th Gen Intel(R) Core(TM) i7-13650HX, Linux 7.0.0-38-generic, Python 3.14.4, PyCryptodome 4.2.0. Script `backend/benchmark.py` calls the same functions as the API on random data (cipher speed does not depend on content). Each measurement has one warm-up run, then 15 repetitions (cipher-only) or 5 (end-to-end, which includes PBKDF2); we report the median, which is robust to background load.

3.3. Results

Evidence type	Size	AES-256-CTR	DES-CTR	RC4
Chat screenshot	0.5 MB	0.35 / 117	17.7 / 134	1.31 / 118
Identity scan	1 MB	0.65 / 118	34.8 / 152	2.57 / 120
Office document	2 MB	1.25 / 123	70.4 / 194	5.02 / 128
Video recording	50 MB	44.7 / 449	1743 / 2170	145 / 524
Video (upload limit)	100 MB	89.9 / 771	3481 / 4235	286 / 942

Table 2: Median encryption time in ms: cipher only / end-to-end upload processing (incl. PBKDF2 114 ms, HMAC, Base64/JSON). Throughput (cipher only): AES 1112, DES 28.7, RC4 349 MB/s. Decryption times are similar.

3.4. Interpretation

For the evidence a victim uploads most often (screenshots, scans, documents up to 2 MB) the cipher does not matter to the user: PBKDF2 alone takes 114 ms, i.e. 98 % of the 117 ms AES upload of a screenshot, and even DES adds at most 70.4 ms. This cost is deliberate: it is what makes guessing a reporter's passphrase from a stolen envelope expensive. The cipher matters only for video. At the 100 MB limit DES is 38.7 × and RC4 3.2 × slower than AES, so an upload takes 4.23 s with DES against 0.77 s with AES. That RC4 is slower than AES on this machine is most likely due to the CPU's AES-NI instructions, which PyCryptodome can use; we did not test without them. For AES on large files, Base64/JSON and HMAC cost more than the encryption itself (567 ms vs. 89.9 ms), and they also set the storage cost: every envelope is 1.33 × the file for all three ciphers, because CTR and RC4 add no padding. We therefore recommend AES-256-CTR with HMAC: it is the fastest here and the only one of the three still considered secure; DES and RC4 are kept because the assignment requires them.

4. Limitations and Conclusion

The main gap for this scenario is that the incident notes, filename and category are stored in plaintext and are not covered by the MAC; they should move inside the encrypted, authenticated part. Also: decryption checks login but not ownership, so access rests on the passphrase alone, and reporters keep their envelope (they cannot fetch it from the server); the `admin` role goes to whoever first registers "admin"; sessions live in server memory and the token in localStorage (readable by XSS); the server sees plaintext, so HTTPS and a trusted host are required; DES's 56-bit key is brute-forceable and RC4 is biased. Out of scope: a compromised server or reporter device. In production we would use AES only (or AES-GCM), encrypt all metadata, stream large files and store binary ciphertext. CipherVault shows that evidence files of sexual violence can be stored so that a leaked database or envelope does not reveal their content without the reporter's passphrase, and that with AES this costs a victim about 0.12 s for a typical document and under one second for a 100 MB video.

AI usage. ChatGPT: https://chatgpt.com/share/6ae05665-e3d4-83ec-832e-7b93d76c83d1 — *[TODO: what the group used ChatGPT for]*. Claude Code (Anthropic) was used to redesign the frontend, merge the CTR/HMAC changes into it, write `backend/benchmark.py` and draft this report; log: *[TODO: shared link to the Claude log]*. *[TODO: How the group checked the AI output (e.g. re-ran the benchmark, compared claims with the code and the cited regulations).]*